JournaBuddy

System Architecture, Planning, Implementation & Visuals Report

Comprehensive "For Dummies" Explanatory Guide

Abdullah Al Mamun
BSc, MSc - Software Engineering
TU Wien (Vienna, Austria) & Daffodil International University
Email: mamun.swe.de@gmail.com — GitHub: @abbysweb
ORCID: 0009-0006-7473-0024

July 29, 2026

Contents

1 Introduction: What is JournaBuddy?

JournaBuddy is an artificial intelligence platform designed to help scientific authors prepare, optimize, and evaluate their research papers before submitting them to academic journals.

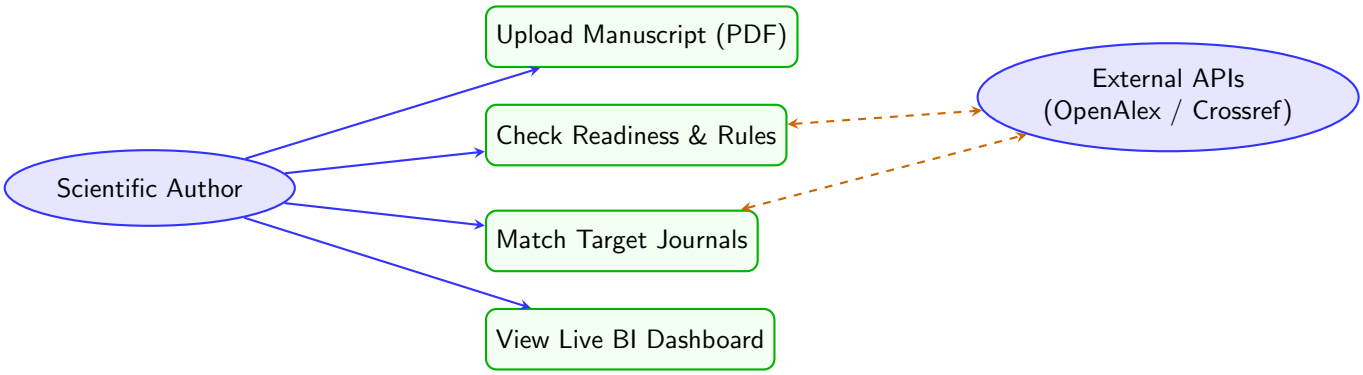
Instead of guessing whether a manuscript will be rejected due to poor formatting, weak citations, or out-of-scope journal selection, JournaBuddy runs automated checks that evaluate the paper across three core layers:

1. **Symbolic / Rule-Based Layer:** Checks exact facts (structure, acronym definitions, citation formatting).
2. **Statistical NLP Layer:** Calculates mathematical text scores (readability, passive voice, jargon density).
3. **Semantic / AI Layer:** Uses local LLM agents and vector search to evaluate methodology and journal scope fit.

2 System Diagrams & Visual Architecture

2.1 1. Use Case Diagram

The Use Case Diagram shows all primary interactions between scientific authors, external academic services, and the JournaBuddy system.



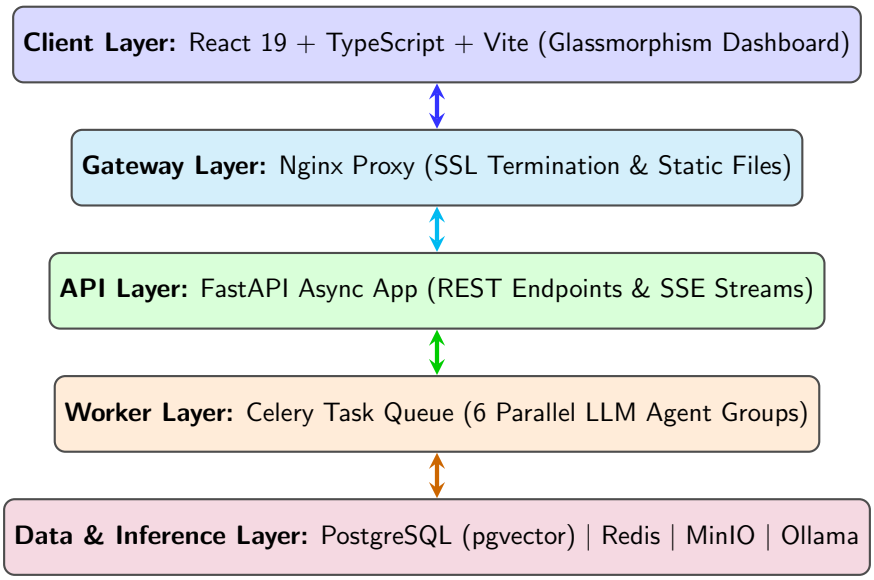
Intuitive Diagram Breakdown: Use Case

Think of this diagram as a service interaction menu:

- **Scientific Author:** Performs actions like uploading manuscripts, checking readiness rules, matching journals, and viewing live dashboard analytics.
- **External Academic APIs:** Query services like OpenAlex and Crossref in the background for citation metrics and publisher trust indicators.

2.2 2. System Architecture Diagram

The System Architecture Diagram displays the multi-tier containerized stack powering JournaBuddy.



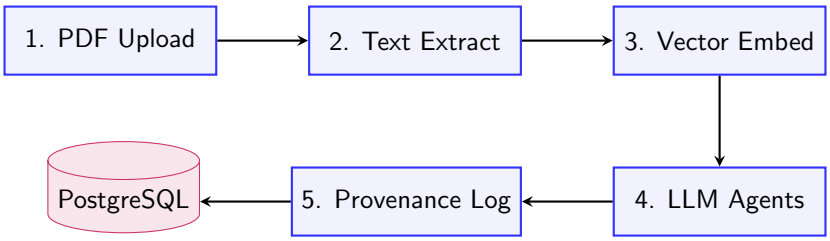
Intuitive Diagram Breakdown: System Architecture

Imagine JournaBuddy as a 5-story containerized stack:

1. **Top Floor (Client):** The visual web interface where authors view charts and upload papers.
2. **4th Floor (Gateway):** Nginx proxy directing traffic and terminating SSL.
3. **3rd Floor (API):** FastAPI server managing file uploads and streaming progress.
4. **2nd Floor (Workers):** Celery workers running parallel analysis tasks.
5. **Basement (Data & Brain):** Vault storing PostgreSQL vectors, Redis queues, MinIO binaries, and local Ollama LLMs.

2.3 3. Data Flow Diagram (DFD)

The Data Flow Diagram tracks how an uploaded paper moves through processing stages and transforms into score reports.



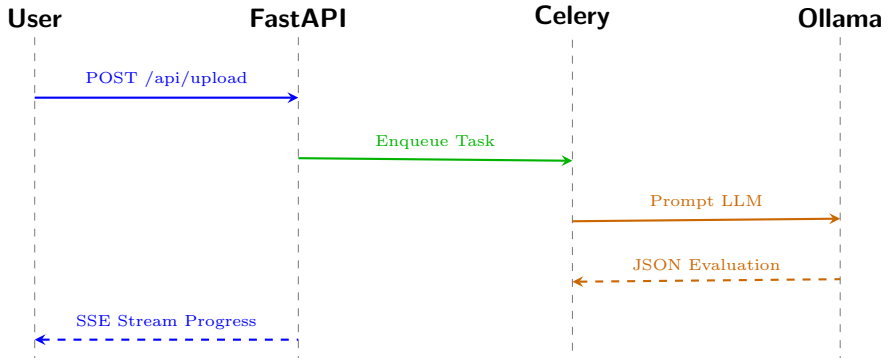
Intuitive Diagram Breakdown: Data Flow

The manuscript moves through 5 distinct processing stages:

1. **PDF Upload:** Raw file received and saved securely in MinIO storage.
2. **Text Extraction:** PDF visual layout parsed into structured raw text.
3. **Vector Embedding:** Text converted into 384-dimensional semantic numerical arrays.
4. **LLM Agents:** Parallel AI workers evaluate tone, methodology, and scope fit.
5. **Provenance Log:** Metric calculation sources recorded in PostgreSQL database.

2.4 4. Sequence Diagram

The Sequence Diagram displays the exact order of network messages between components during a live upload.



Intuitive Diagram Breakdown: Sequence Diagram

This is a timeline showing real-time network messages between components:

1. You click **"Upload"**, sending your manuscript to FastAPI.
2. FastAPI enqueues the job into Celery so your browser doesn't freeze waiting.
3. Celery sends text prompts to local Ollama LLM workers.
4. Ollama returns structured JSON evaluation scores back to Celery.
5. FastAPI streams real-time progress right back to your screen over SSE.

3 Continuous Implementation Roadmap

- **Phase 1 (Infrastructure):** Podman Desktop / Docker Compose, FastAPI, MinIO, Vite React setup. (Status: *Complete & Verified*)
- **Phase 2 (AI Pipeline):** Celery workers, Ollama Llama 3.1 integration, vector embeddings.
- **Phase 3 (External Enrichment):** OpenAlex, Crossref, DOAJ integrations.
- **Phase 4 (Real-time UI):** Glassmorphism dashboard, SSE streaming, interactive charts.
- **Phase 5 (Hardening):** Redis caching, Prometheus/Grafana monitoring, security audits.